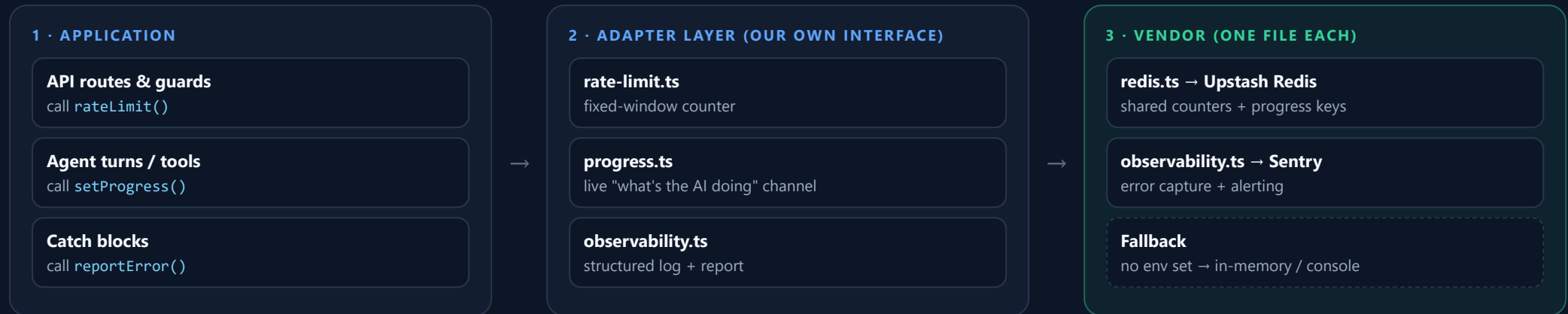


Helix — Shared State & Monitoring Infrastructure

Helix runs as many serverless instances at once. Two needs must be shared across them — request limits and live progress — and failures must be visible. Each capability sits behind a thin adapter so the vendor can change without touching app code.



Rate limiting

Per-instance counters let the real limit multiply by the number of instances. A shared Redis counter (`INCR + EXPIRE`) makes one global limit that actually holds.



Live progress

The agent turn and the client's status poll can land on different instances. Storing progress in Redis (short TTL) means the spinner reflects real work, not a stuck local copy.



Error monitoring

Server errors (failed webhooks, crashed background tasks) were only in unwatched logs. `reportError()` always logs structurally and, when a DSN is set, sends to Sentry with context.



Graceful by default

Every adapter falls back to its in-process behavior when the vendor env vars are absent — so local and single-instance setups run unchanged with zero configuration.

Portability — why the adapter boundary matters

App code never imports Upstash or Sentry directly; it calls the functions above. Each vendor is confined to one file (`redis.ts`, `observability.ts`) carrying a PORTABILITY NOTE. Moving to AWS (ElastiCache / CloudWatch) or Azure (Cache for Redis / Application Insights) means rewriting that one file — Redis is a standard protocol with throwaway data, and Sentry is host-agnostic. No application rewrite, no data migration.